



7a. GloVe & Transformer

Never Stand Still

Faculty of Engineering

COMP9444 Week 5

Hao Xue

School of Computer Science and Engineering

Faculty of Engineering

The University of New South Wales, Sydney, Australia

cs9444@cse.unsw.edu.au

Word2vec Summary

- Go through each word in the whole corpus and
 - predict surrounding words for each word (skip-gram)
 - predict centre word, based on surrounding words (CBOW)
- This captures co-occurrence of words one at a time.

Why not capture co-occurrence counts directly?

Global Vectors (GloVe)

- GloVe stands for Global Vectors. It's a model for learning word embeddings (just like Word2Vec) but it focuses on capturing global co-occurrence statistics from a corpus.
- Recall: In natural language, words that appear in similar contexts tend to have similar meanings.

Global Vectors (GloVe)

- GloVe leverages the **co-occurrence statistics** of words in a corpus to learn the word embeddings.
- **Co-occurrence probabilities:** Given two words i and j that occur in text, their co-occurrence probability is defined as the probability of seeing i in the context of j

$$P(j/i) = \frac{\text{count}(j \text{ in context of } i)}{\sum_k (\text{count}(k \text{ in context of } i))}$$

- If we want to distinguish between two words, it is not enough to look at their co-occurrences, we need to look at the ratio of their co-occurrences with other words.
 - Formalizing this intuition gives us an optimization problem

Global Vectors (GloVe)

Table 1: Co-occurrence probabilities for target words *ice* and *steam* with selected context words from a 6 billion token corpus. Only in the ratio does noise from non-discriminative words like *water* and *fashion* cancel out, so that large values (much greater than 1) correlate well with properties specific to ice, and small values (much less than 1) correlate well with properties specific of steam.

Probability and Ratio	$k = \textit{solid}$	$k = \textit{gas}$	$k = \textit{water}$	$k = \textit{fashion}$
$P(k \textit{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \textit{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \textit{ice})/P(k \textit{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [GloVe](#): Global Vectors for Word Representation

GloVe Objective

Notation:

- i : word, j : a context word
- w_i : The word embedding for i
- c_j : The context embedding for j
- b_i^w, b_j^c : Two bias terms: word and context specific
- X_{ij} : The number of times word i occurs in the context of j

Intuition:

1. Construct a word-content matrix whose $(i, j)^{th}$ entry is $\log(X_{ij})$
2. Find vectors w_i, c_j and the biases b_i, c_j such that the dot product of the vectors added to the biases approximates the matrix entries

$$w_i^\top c_j + b_i^w + b_j^c \approx \log(X_{ij})$$

learn word vectors such that their similarity (dot product) approximates how often they co-occur

GloVe Objective

Notation:

- i : word, j : a context word
- w_i : The word embedding for i
- c_j : The context embedding for j
- b_i^w, b_j^c : Two bias terms: word and context specific
- X_{ij} : The number of times word i occurs in the context of j

Objective:

$$J(\theta) = \sum_{i,j=1}^{|V|} (w_i^T c_j + b_i + b_j - \log X_{ij})^2$$

Problem: Pairs that frequently co-occur tend to dominate the objective

Solution: Correct for this by adding an extra term that prevents this (Adding a weighting function)

GloVe Objective

Objective:

$$J(\theta) = \sum_{i,j=1}^{|V|} f(X_{ij})(\mathbf{w}_i^T \mathbf{c}_j + b_i + b_j - \log X_{ij})^2$$

Learn word vectors such that their dot product equals the logarithm of the words' probability of co-occurrence,
i.e., For each pair of words i, j , minimise distance between **dot product** and overall **log count** in corpus

GloVe vs Word2Vec

GloVe (Count-based)

- Fast training (if corpus is not too large)
- Efficient use of statistics
- Primarily used to capture word similarity
- Good performance with less dimensionality and even less corpora

Word2Vec (Prediction-based)

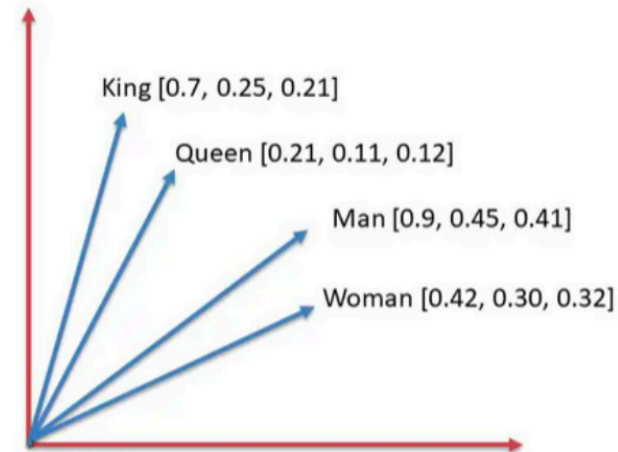
- Scales with corpus size
- Inefficient use of statistics
- Can capture complex patterns beyond word similarity
- Improved performance on extrinsic tasks

Linguistic Regularities

King + Woman - Man $\simeq$ Queen

More generally,
A is to B as C is to ??

$$d = \operatorname{argmax}_x \frac{(v_c + v_b - v_a)^T v_x}{\|v_c + v_b - v_a\|}$$



$$\begin{aligned} \text{King} - \text{Man} + \text{Woman} &= [0.7, 0.25, 0.21] - [0.9, 0.45, 0.41] + [0.42, 0.30, 0.32] \\ &= [0.22, 0.1, 0.12] \approx \text{Queen} \end{aligned}$$

Word Analogy Task

Q1. **evening** is to **morning** as **dinner** is to

- A. breakfast
- B. soup
- C. coffee
- D. time

Q2. **bow** is to **arrow** as is to **bullet**

- A. defend
- B. lead
- C. shoot
- D. gun

Word Analogies

Type of relationship	Word Pair 1		Word Pair 2	
Common capital city	Athens	Greece	Oslo	Norway
All capital cities	Astana	Kazakhstan	Harare	Zimbabwe
Currency	Angola	kwanza	Iran	rial
City-in-state	Chicago	Illinois	Stockton	California
Man-Woman	brother	sister	grandson	granddaughter
Adjective to adverb	apparent	apparently	rapid	rapidly
Opposite	possibly	impossibly	ethical	unethical
Comparative	great	greater	tough	tougher
Superlative	easy	easiest	lucky	luckiest
Present Participle	think	thinking	read	reading
Nationality adjective	Switzerland	Swiss	Cambodia	Cambodian
Past tense	walking	walked	swimming	swam
Plural nouns	mouse	mice	dollar	dollars
Plural verbs	work	works	speak	speaks

Word Relationship

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Summary

- Word vectors, also sometimes called word embeddings or word representations are distributed representations of words.
- Two kinds of embeddings
 - Sparse vectors: Words are represented by a simple function of the counts of nearby words.
 - Dense vectors: Representation is created by training a classifier to distinguish nearby and far-away words
- The contexts in which a word appears tells us a lot about what it means. Distributional similarities use the set of contexts in which words appear to measure their similarity
- Word2Vec and GloVe are two important dense representations of words.
- Various choice:
 - dimensionality of embeddings (50, 100, 200, 300, 500)
 - scale, quality and type of text to get word embeddings
 - size of the context window

Attention Mechanism

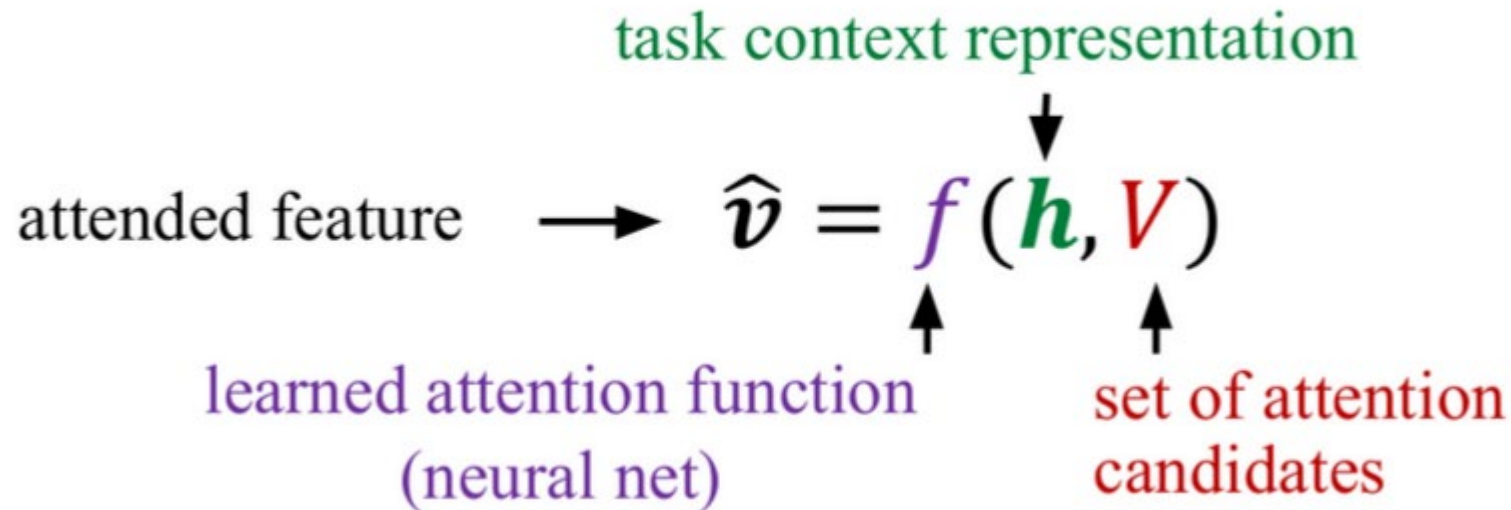
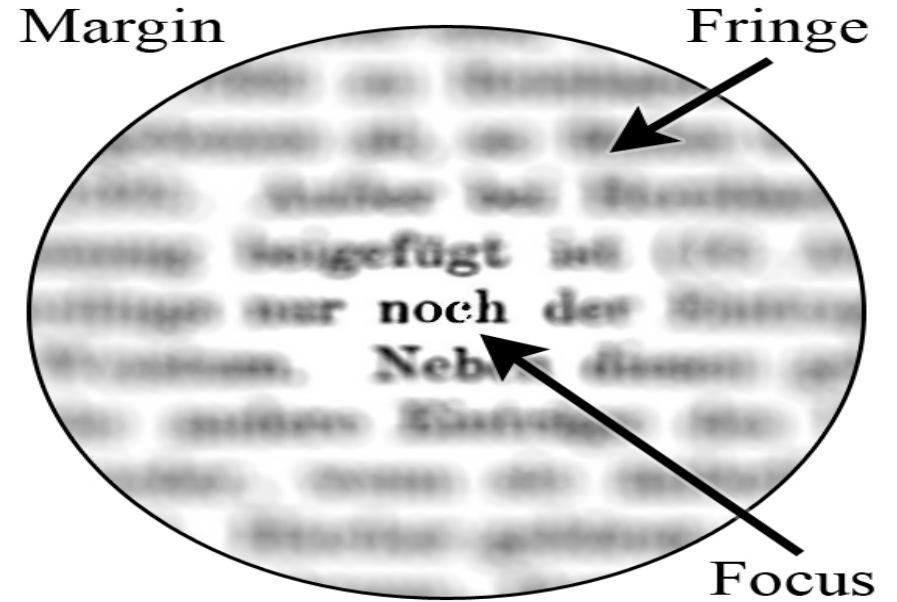


Attention

- Example 1
 - The animal didn't cross the street because it was too tired
 - “it” = ?
- Example 2
 - Once I got home, I ate a very delicious apple.
 - Once I got home, I contacted my friend using Apple.

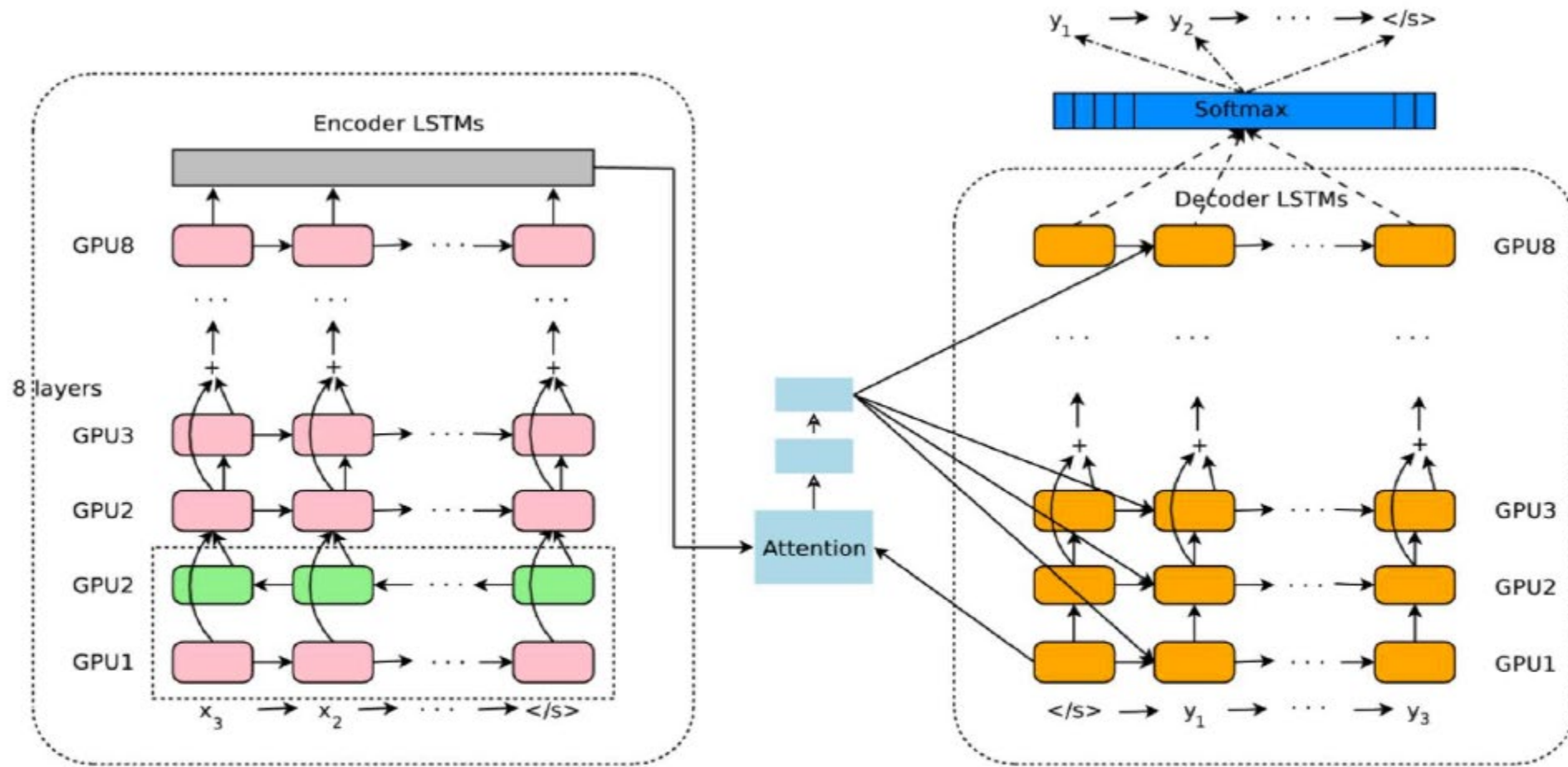
Attention Mechanism

- A set of mechanisms that limit some processing to a subset of incoming stimuli (reducing computational demands)
- Attention in neural networks
 - A mechanism that **learns to focus** on a subset of the **input** that is **relevant to the task**.



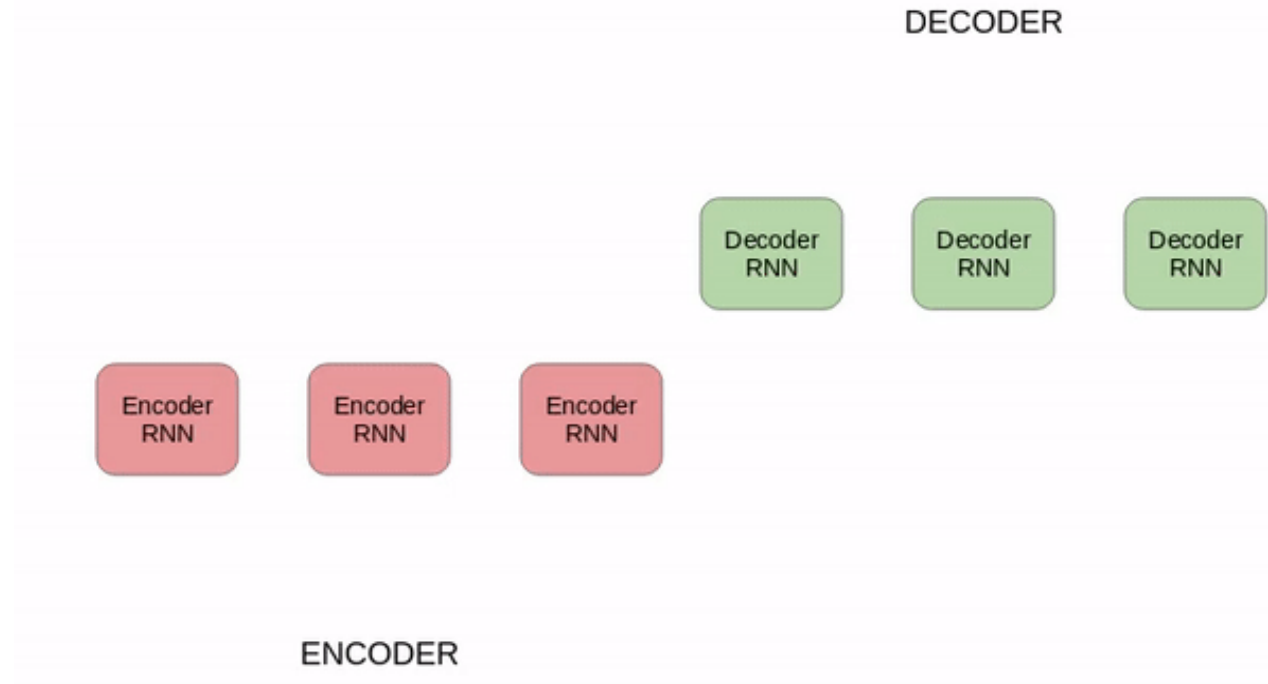
Seq2Seq used for various NLP applications

- Google's Multilingual Neural Machine Translation



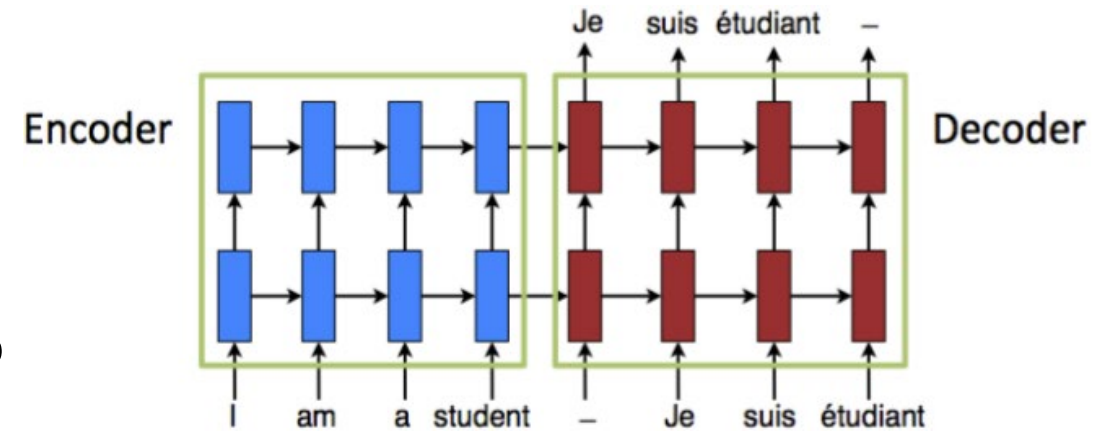
Issues with RNNs for seq2seq tasks

- Despite being very successful for various NLP tasks, there were challenges:
 - dealing with long-range dependencies
 - the sequential nature of the architecture prevents parallelization
- Attention mechanism helped to overcome first issue to certain extent



Neural Machine Translation by jointly Learning to Align and Translate

- NMT attempts to build and train a single, large neural network that reads a sentence and outputs a correct translation.
- The NMT system is a seq2seq model (Encoder-Decoder framework)
- Issue: The encoder-decoder framework compresses all the necessary information of a source sentence into a fixed-length vector.
- Cho et al. (2014) showed that the performance of a basic encoder-decoder deteriorates as the length of an input sentence increases.
- Bahdanau et al. (2015) proposed an extension to NMT which learns to align and translate jointly.
- The basic idea of “attention” is that at each time the model generates a word in a translation, it searches for a set of positions in a source sentence where the most relevant information is concentrated
- The model predicts a target word based on the context vectors associated with these source positions and all the previous generated target words



Neural Machine Translation by jointly Learning to Align and Translate

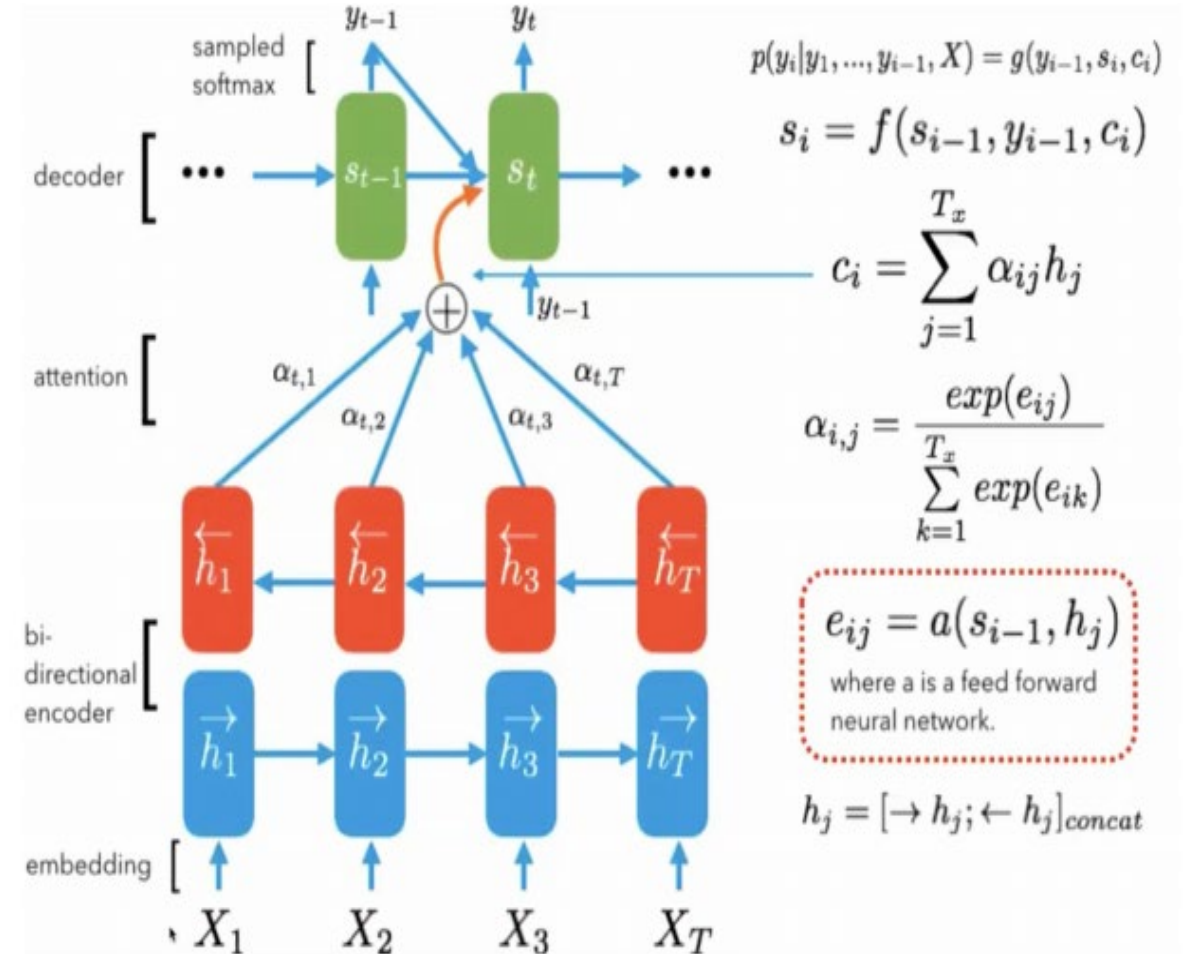
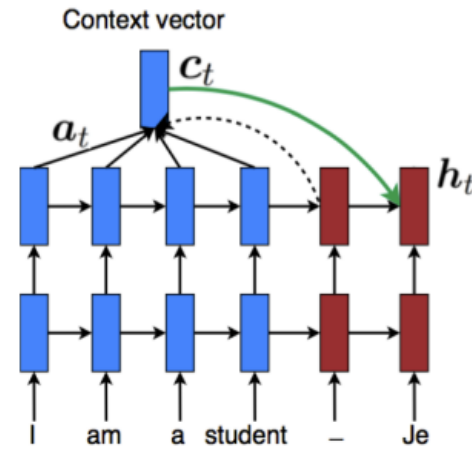
- Solution: Enable the network to pay attention to specific areas of the input by adding new (weighted) connections

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})},$$

$$e_{ij} = a(s_{i-1}, h_j)$$

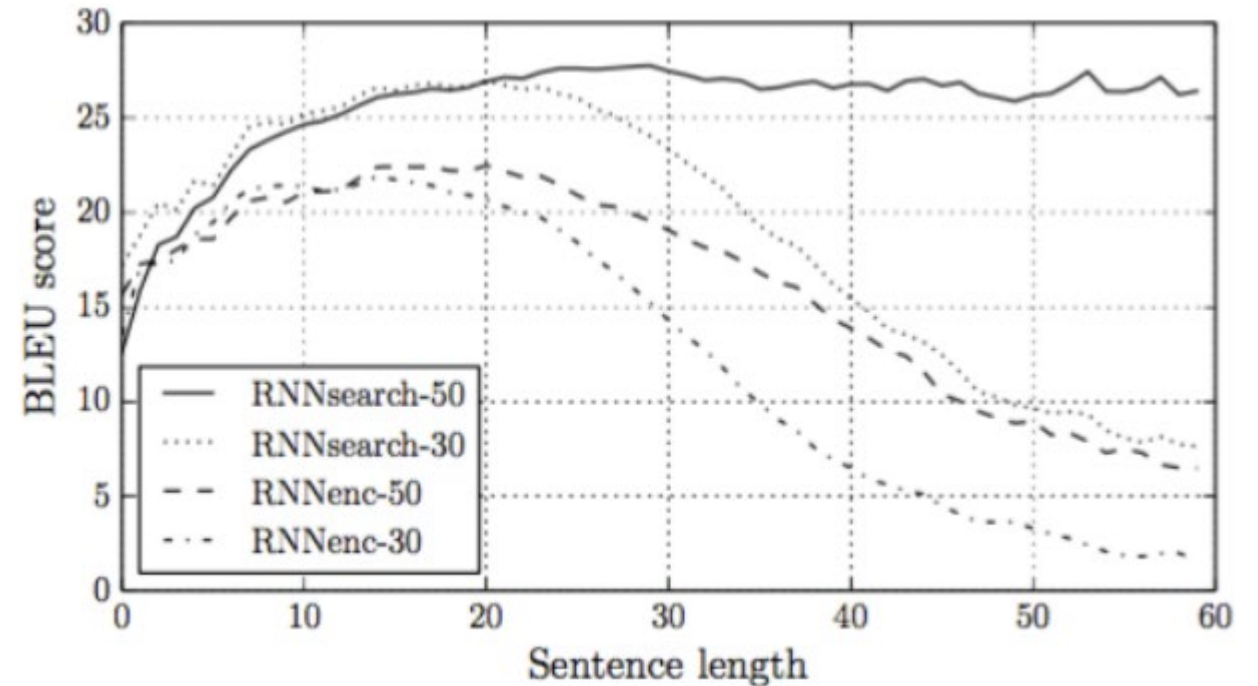
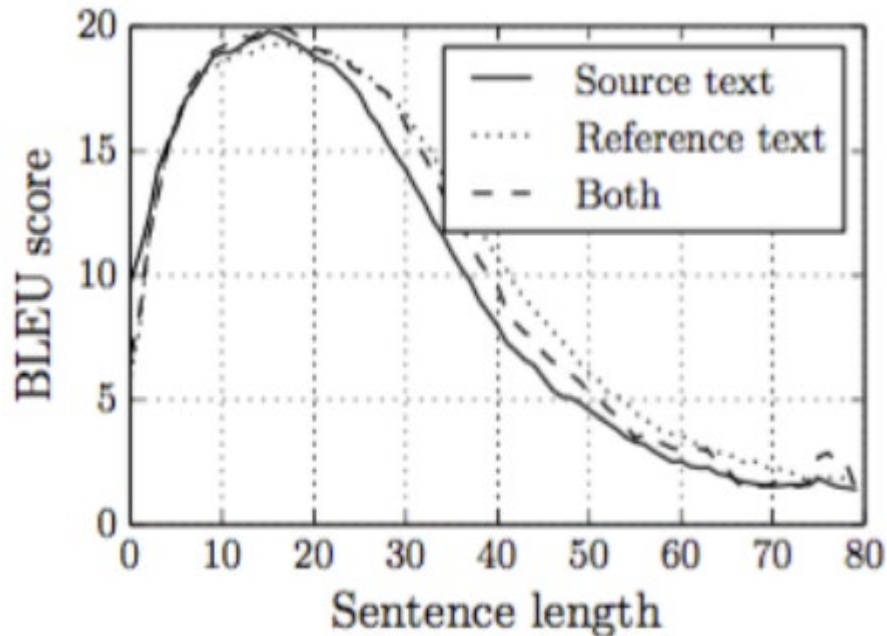
$$\text{score}(h_t, \bar{h}_s) = \begin{cases} h_t^\top \bar{h}_s & \text{dot} \\ h_t^\top W_a \bar{h}_s & \text{general} \\ v_a^\top \tanh(W_a [h_t; \bar{h}_s]) & \text{concat} \end{cases}$$



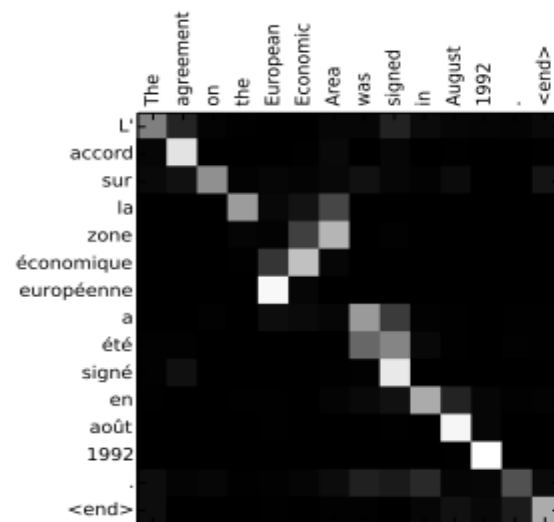
NMT by jointly Learning to Align and Translate

Before Attention: Long sentences are very hard as they are compressed to a fixed length vector

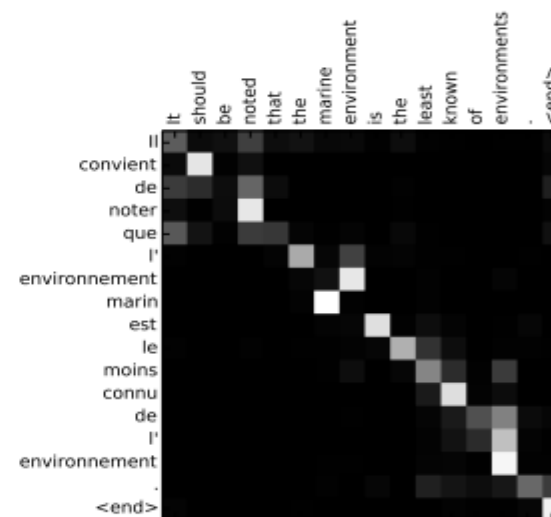
After Attention: The attention mechanism helps to overcome the issue



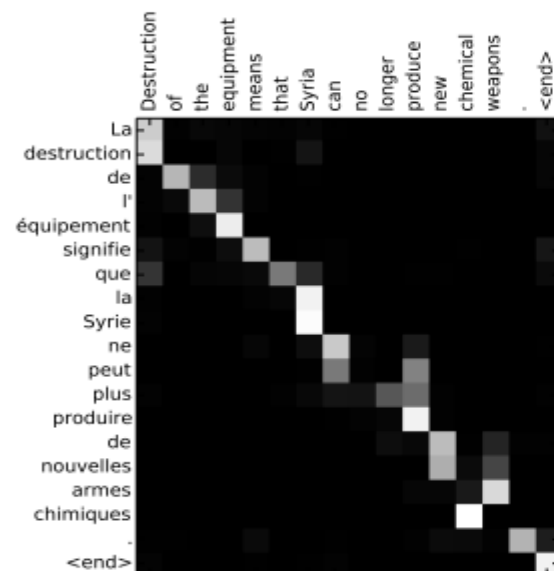
NMT by jointly Learning to Align and Translate



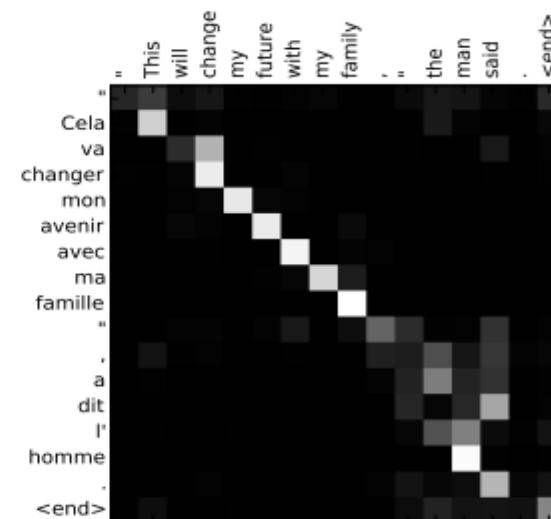
(a)



(b)



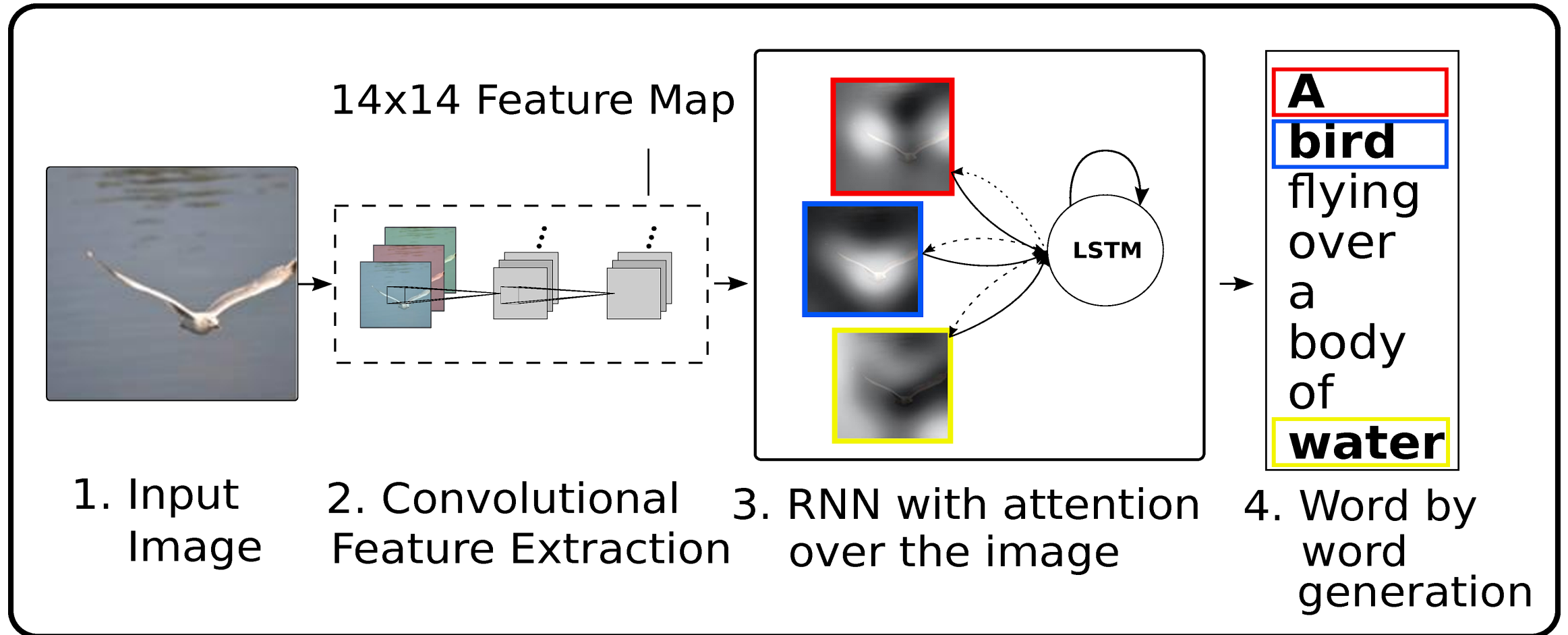
(c)



(d)

Show, Attend and Tell (Xu et al., 2015)

- “Rather than compressing an entire image into static representation, attention allows for **salient features** to dynamically come to forefront as needed”



Show, Attend and Tell (Xu et al., 2015)



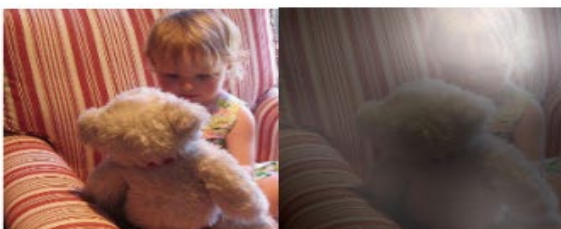
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

<start>



a



large



airplane



flying



in



the



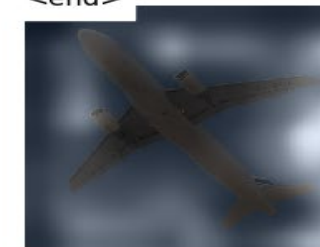
blue



sky



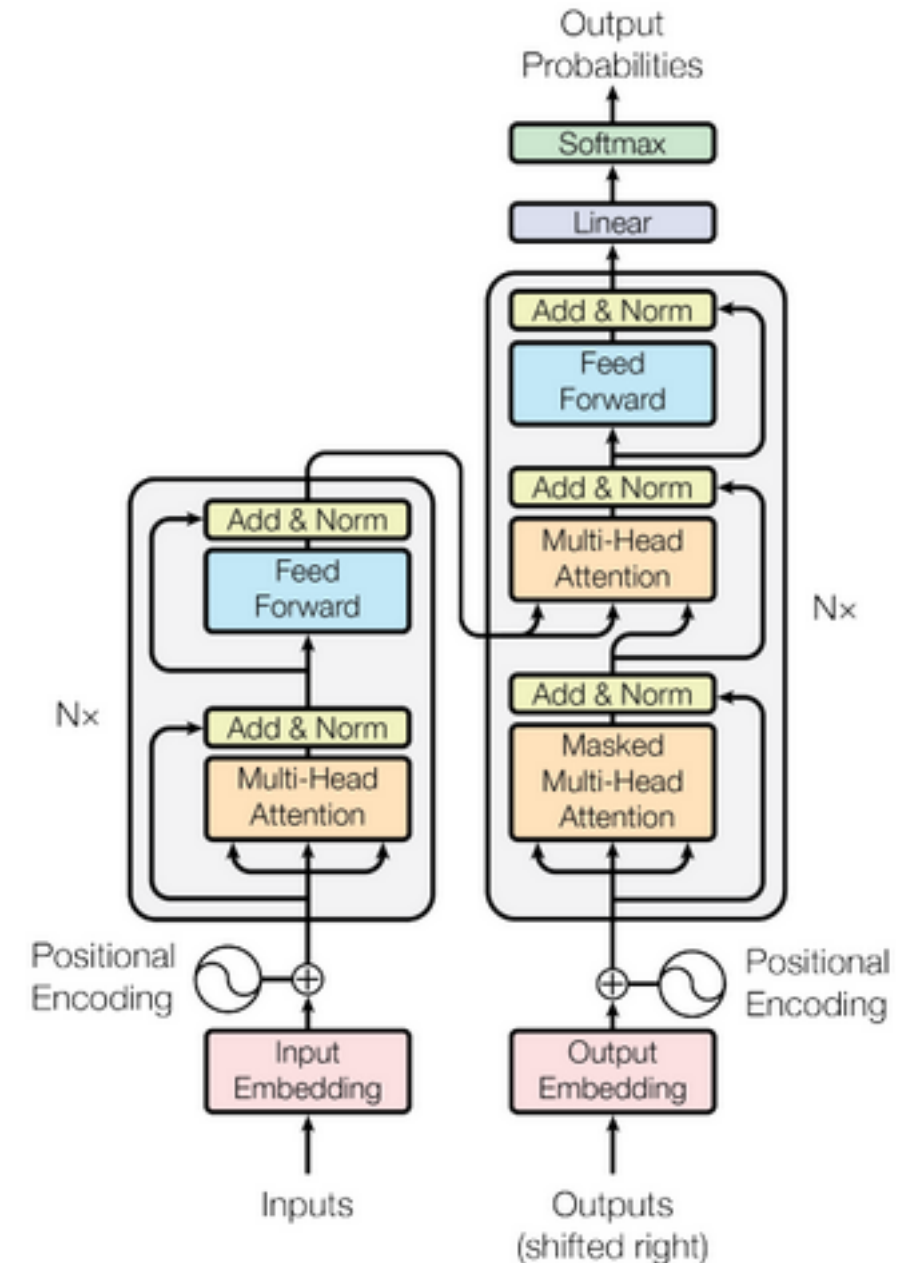
<end>



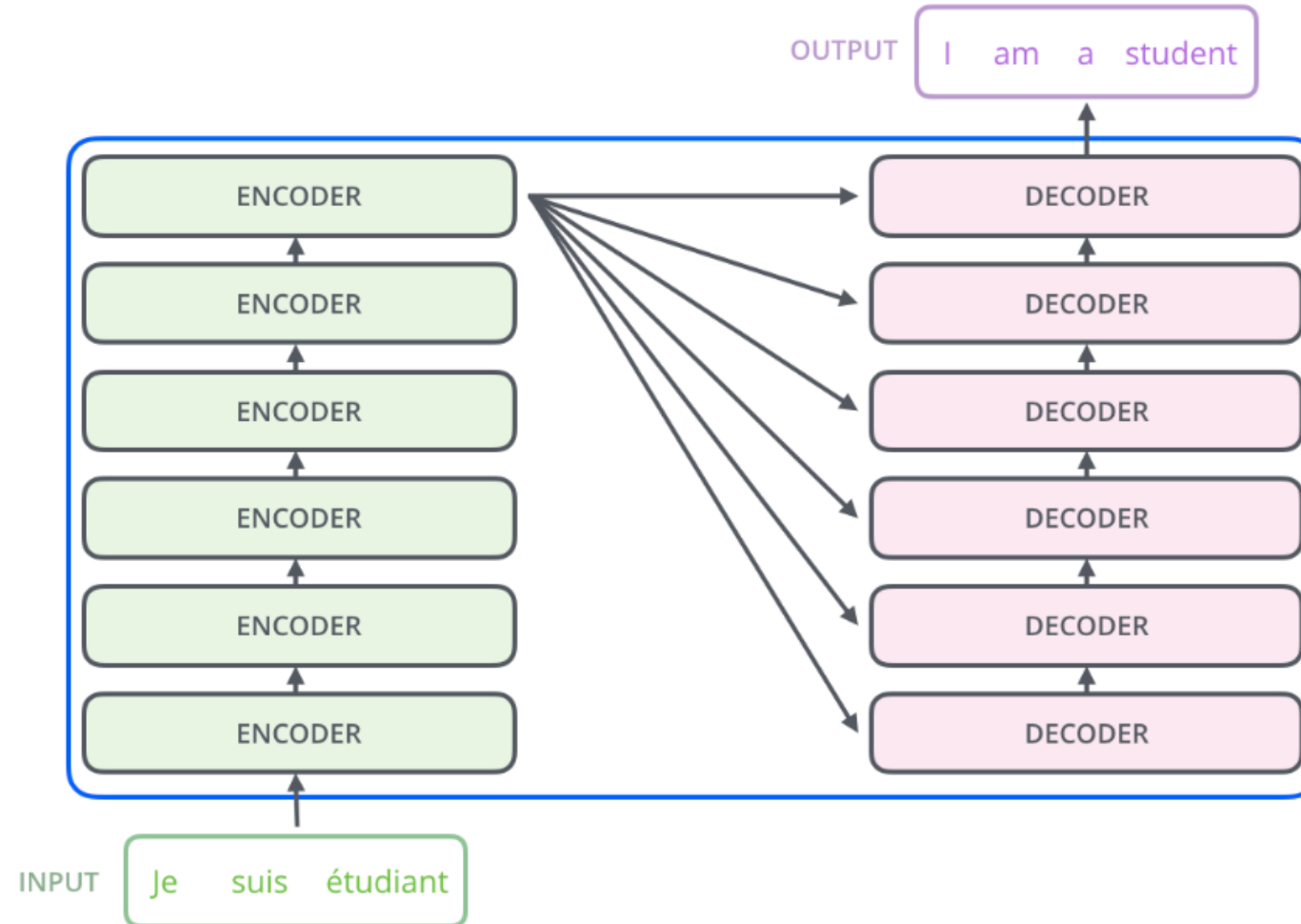
Transformers

- Attention is All You Need
- Novel architecture relies entirely on **self-attention** to compute representations of its input and output without using sequential RNNs or convolutions.
- Aim is to solve seq2seq tasks while handling long-range dependencies

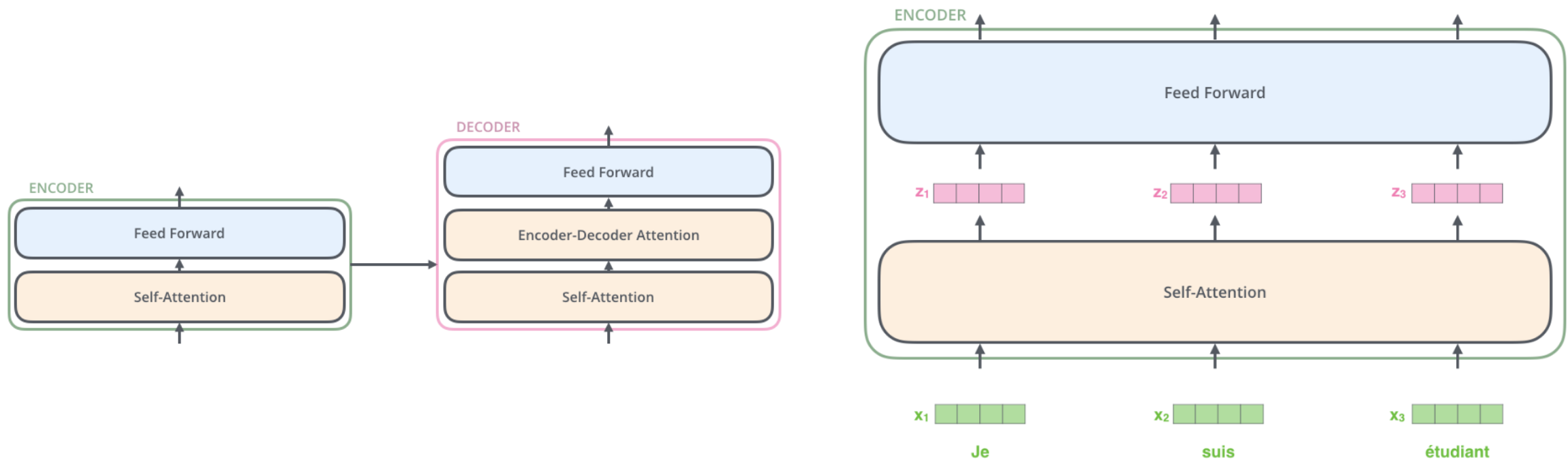
“Griezmann’s announcement comes as a bit of a shock. After enduring the drama surrounding his potential last summer, many thought he was committed to Atletico for more than a year, but the Frenchman seems to have changed his mind.”



Transformer



Transformer



Self-attention in transformers

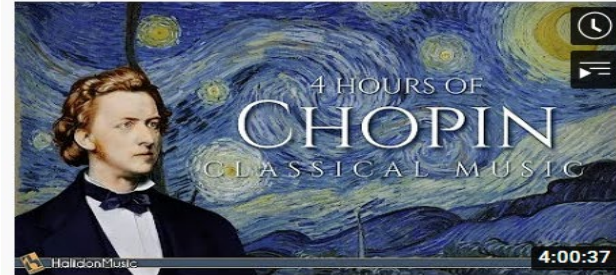
➤ Understanding self-attention with Search

- Query (Q)
- Key (K)
- Value (V)

Value (V)

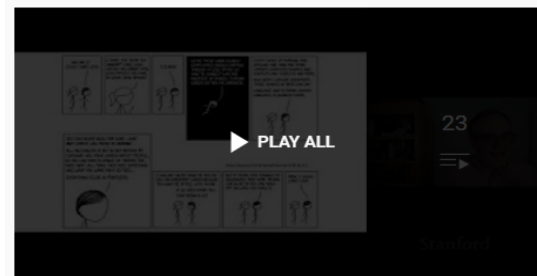


Query (Q)



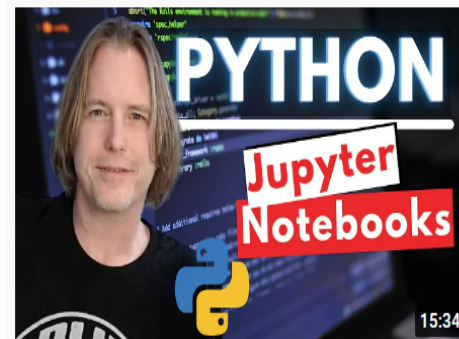
4 Hours Chopin for Studying, Concentration & Relaxation

Key 1 (K1)



Stanford CS224N: Natural Language Processing with Deep Learning | Winter 2021

Key 2 (K2)



Jupyter Notebook Tutorial for Beginners with Python

Key 3 (K3)

98K views • 1 year ago



Dave Gray

This Jupyter Notebook Tutorial for Beginners with Python provides a set up, introduction, and quick start for Jupyter Notebooks.



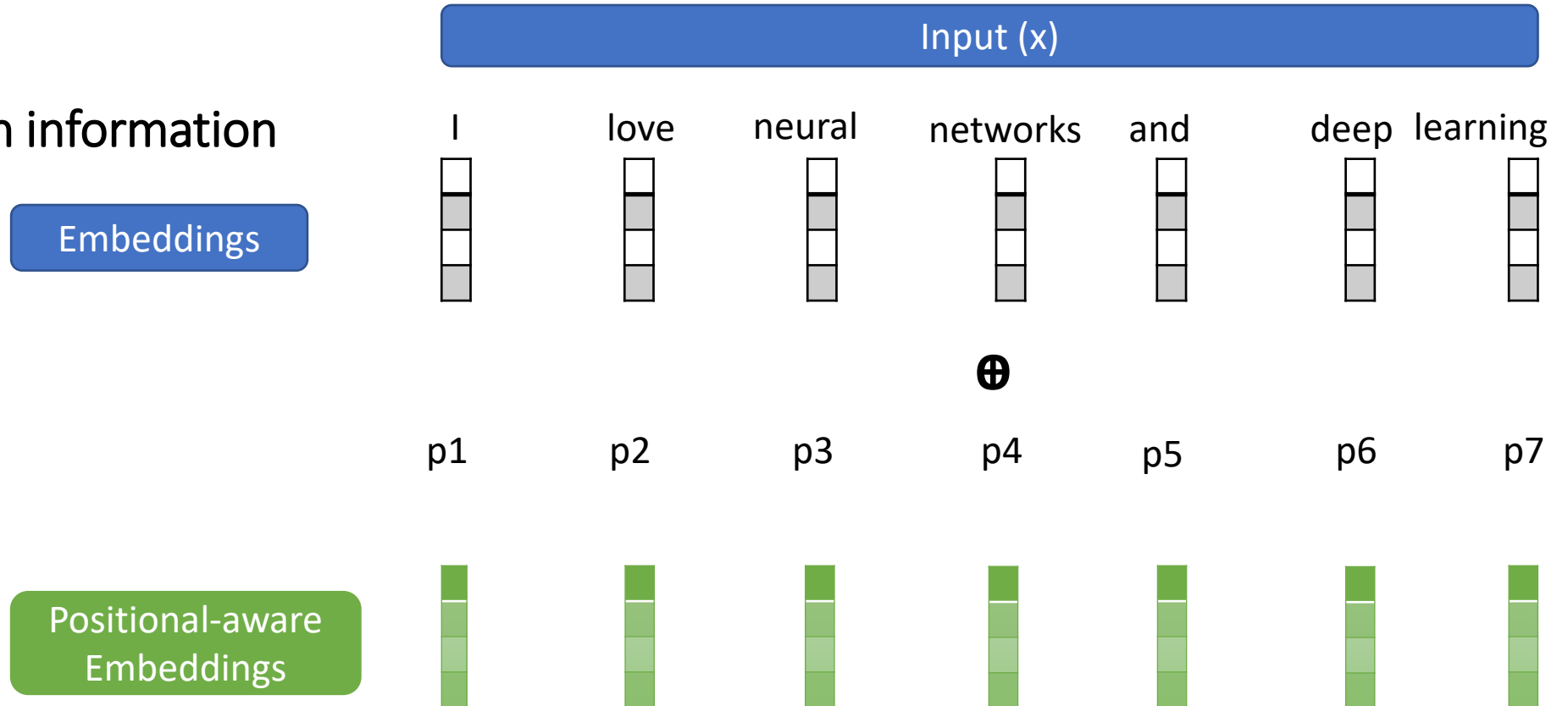
Intro | Download and Install Anaconda | Launch Jupyter Notebook | CLI inside Jupyter Notebook | ... 10 chapters

Self-attention in transformers

Self-attention:

Identify and attend to most important features in input

Step 1: Encode position information



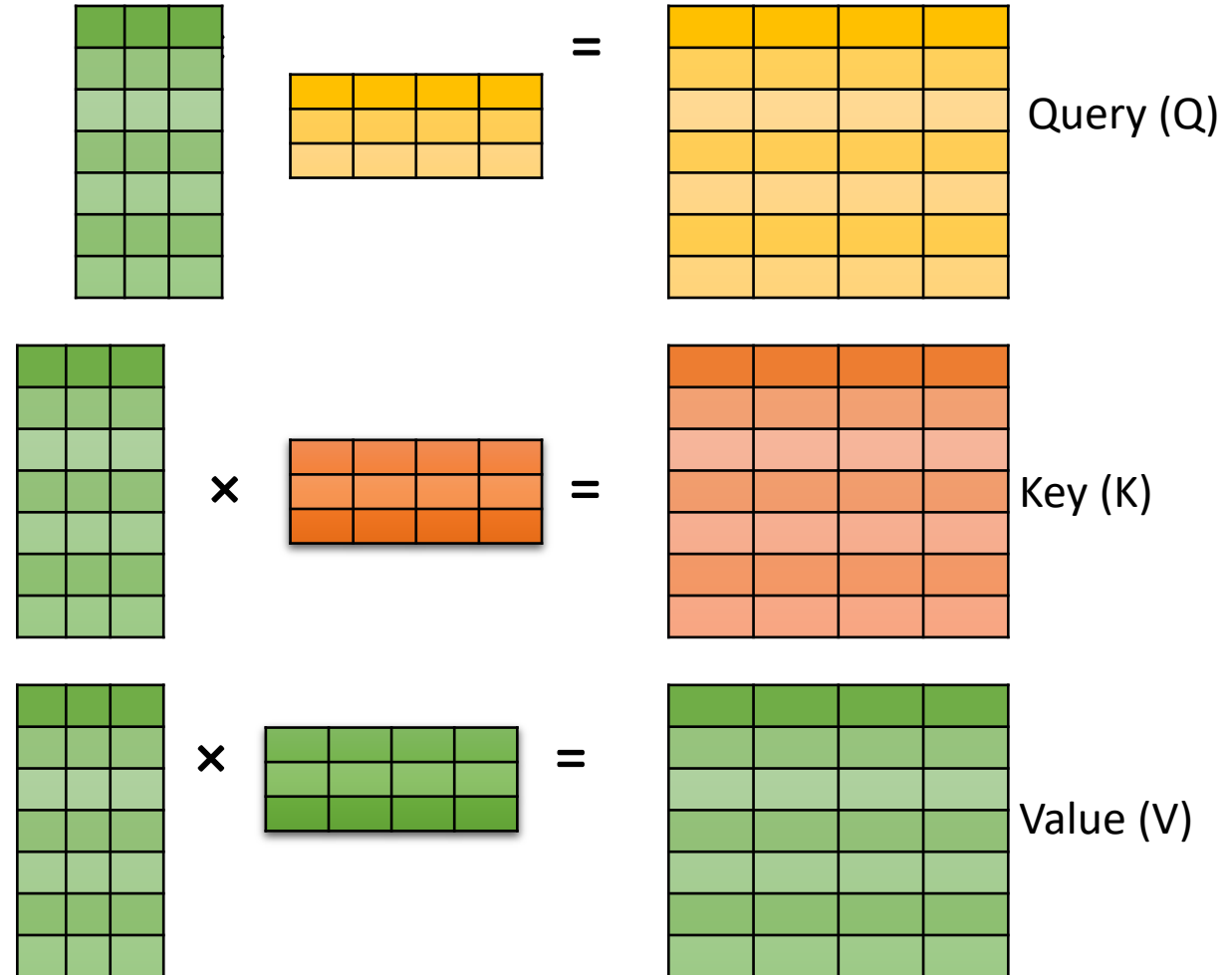
Self-attention in transformers

Self-attention:

Identify and attend to most important features in input

Step 1: Encode position information

Step 2: Extract query (Q), key (K), and value (V)



Self-attention in transformers

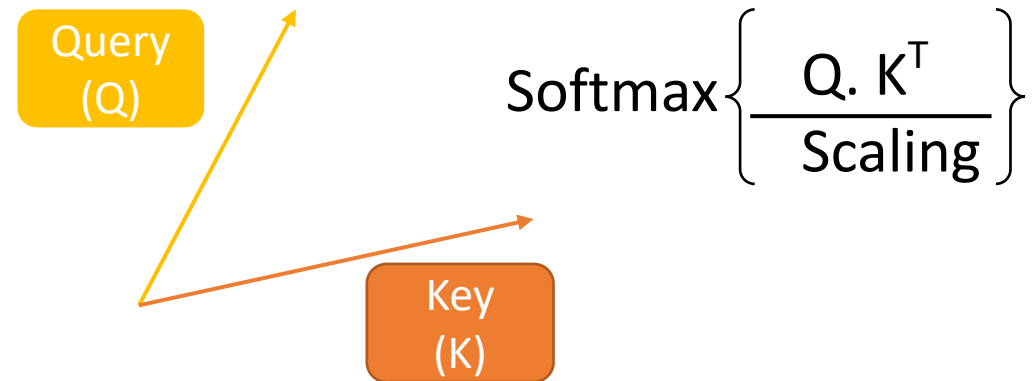
Self-attention:

Identify and attend to most important features in input

Step 1: Encode position information

Step 2: Extract query (Q), key (K), and value (V)

Step 3: Compute attention weighting



Self-attention in transformers

Self-attention:

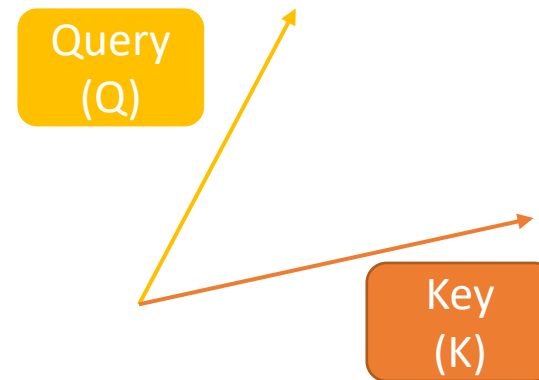
Identify and attend to most important features in input

Step 1: Encode position information

Step 2: Extract query (Q), key (K), and value (V)

Step 3: Compute attention weighting

Step 4: Extract features with high attention



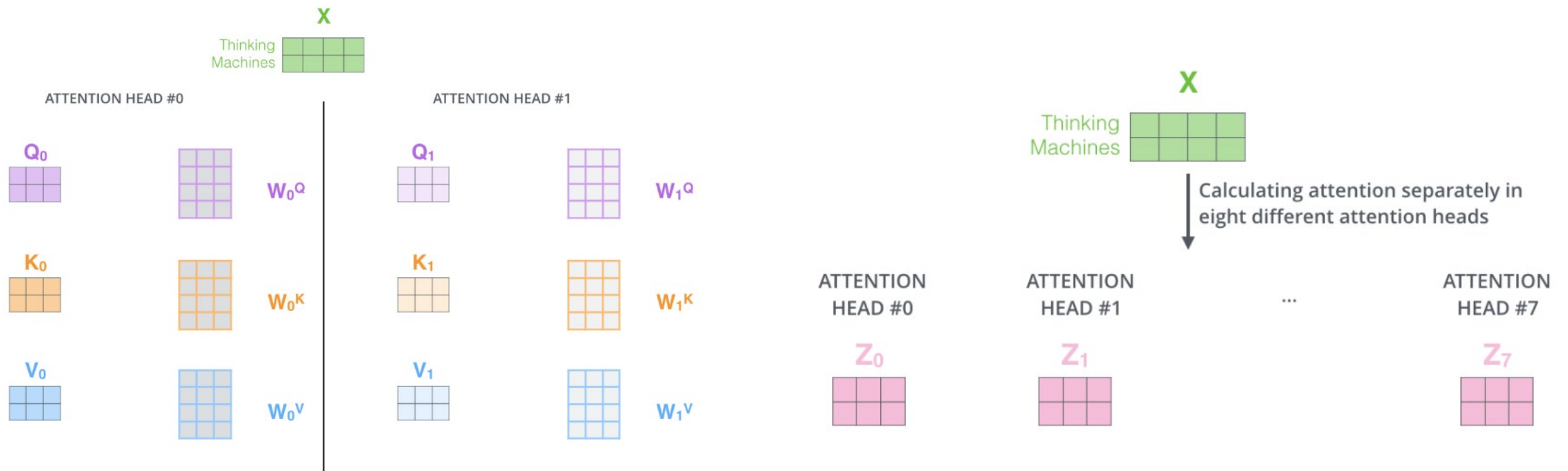
$$\text{Softmax} \left\{ \frac{Q \cdot K^T}{\text{Scaling}} \right\} \times V = A(Q, K, V)$$

Multi-head Attention

Instead of using a single attention mechanism, we use multiple parallel attention “heads”, each with its own learned linear projections.

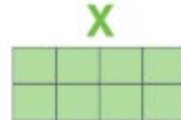
- Each head learns to focus on different parts of the input.
- Multiple Representation Subspaces:
 - Each head uses different projection matrices
 - So attention is computed in **different subspaces**.

Multi-head Attention

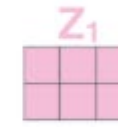
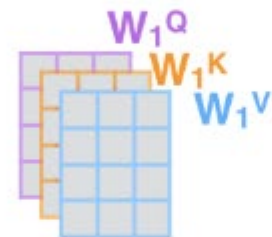
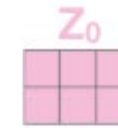
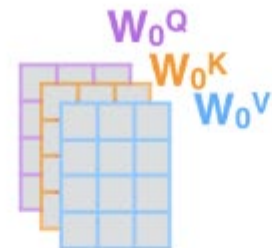
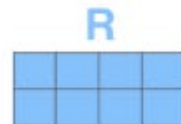


Self-Attention

Thinking
Machines



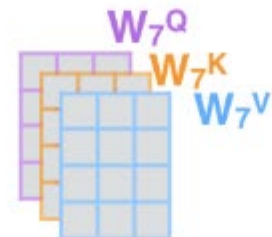
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



...

...

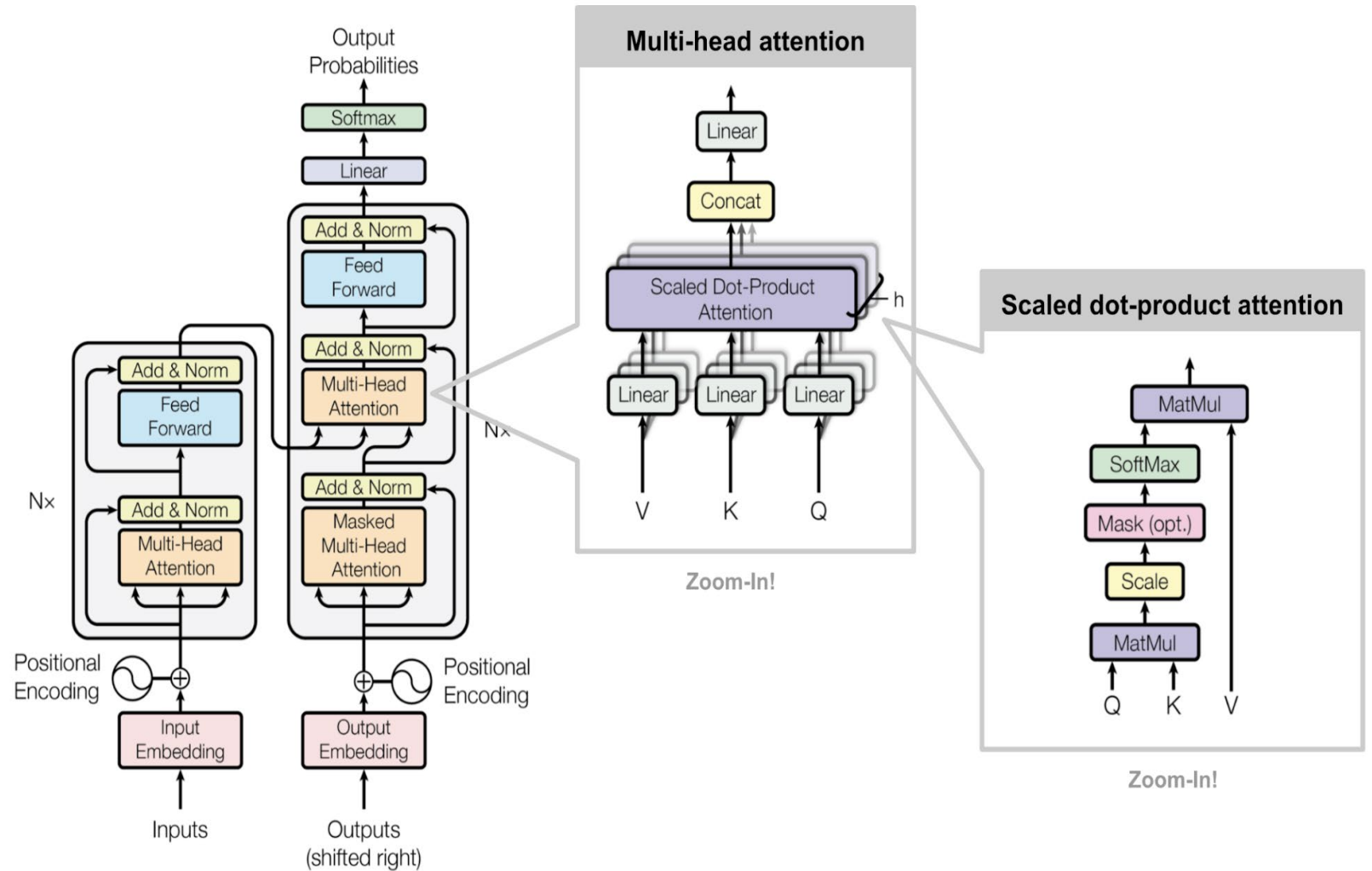
...



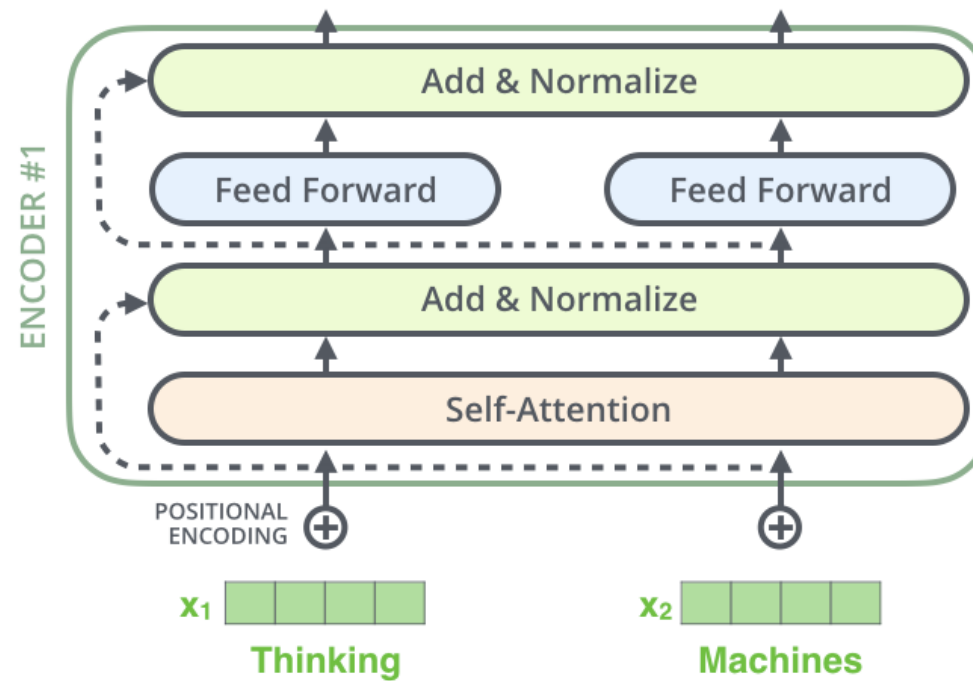
Self-attention in transformers

Self-attention:

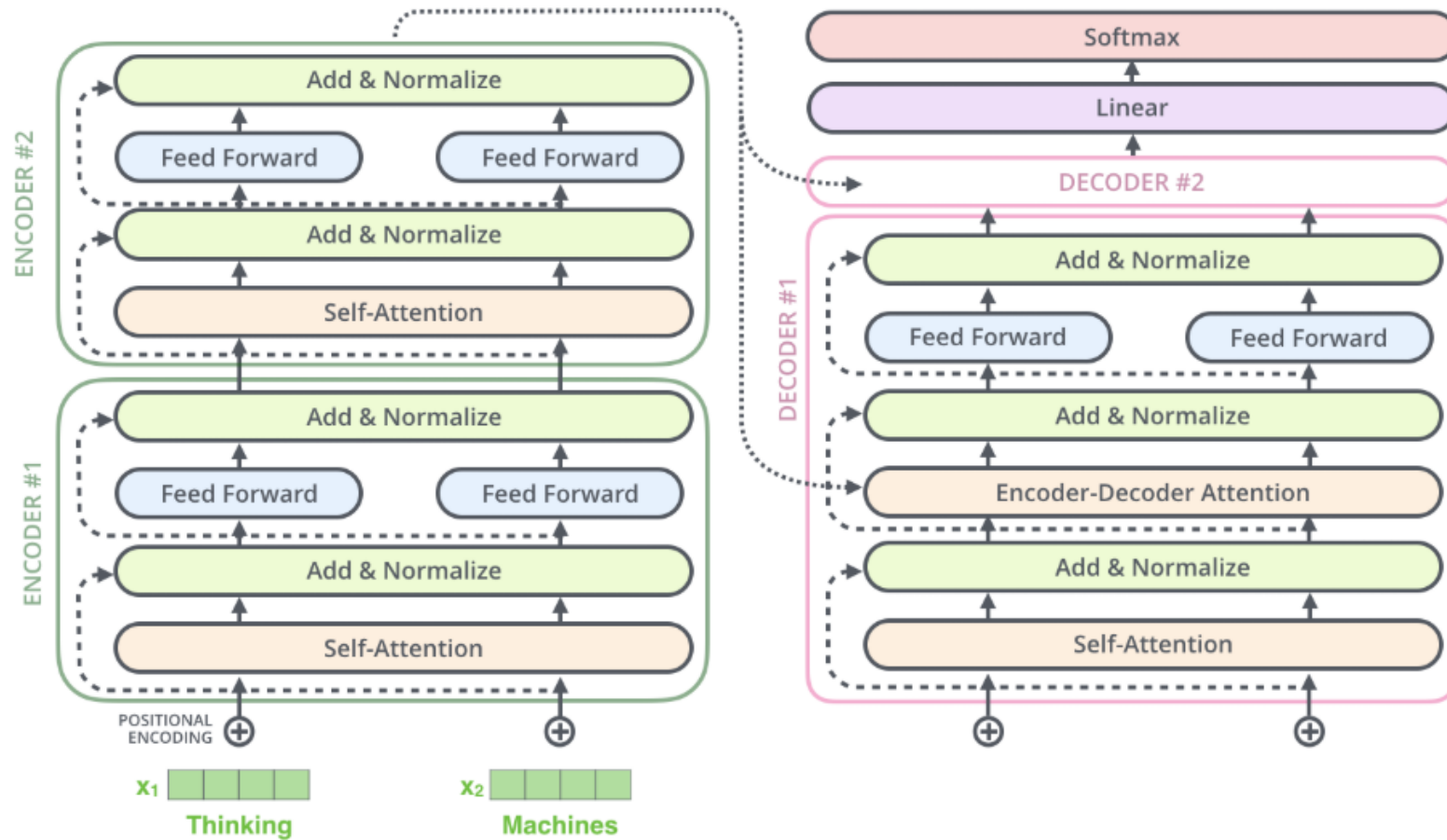
- Step 1: Encode position information
- Step 2: Extract query (Q), key (K), and value (V)
- Step 3: Compute attention weighting
- Step 4: Extract features with high attention



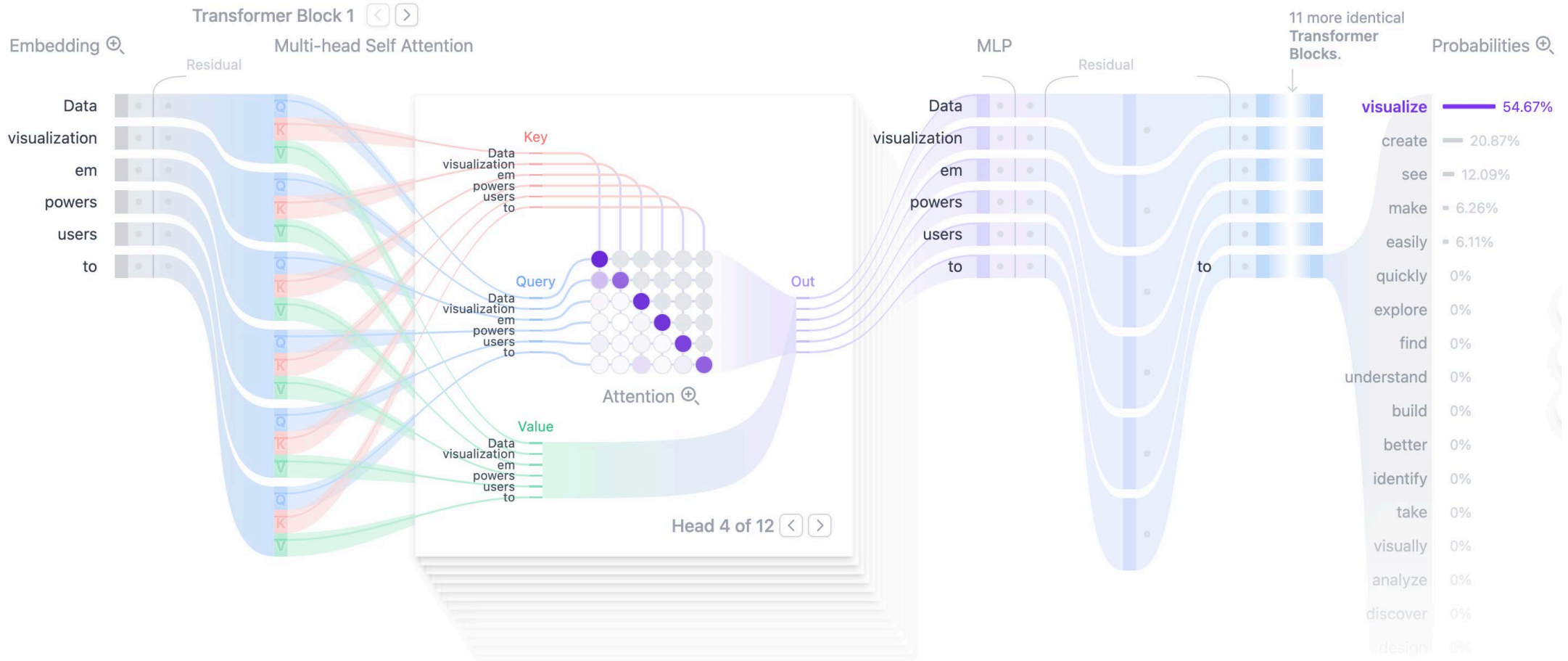
Encoder-Decoder



Encoder-Decoder



Visualization Demo



<https://poloclub.github.io/transformer-explainer/>

Useful Links & Reference

- <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>
- <https://jalammar.github.io/illustrated-transformer/>
- <https://poloclub.github.io/transformer-explainer/>
- <https://poloclub.github.io/dodrio/>

Reference

- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Distributed representations of words and phrases and their compositionality. In Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2 (NIPS'13). Curran Associates Inc., Red Hook, NY, USA, 3111–3119.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global Vectors for Word Representation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- Xin Rong. 2014. word2vec Parameter Learning Explained, [arXiv link](#).
- GloVe <https://nlp.stanford.edu/projects/glove/>
- [On Word Embeddings](#)
- [Word Embedding Demo: Tutorial](#)